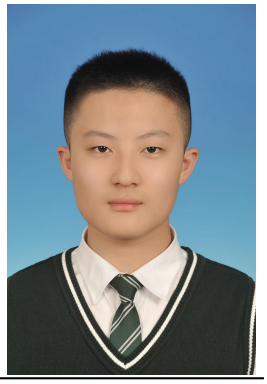
Computational Physics Hw_6 Report

Student ID: 2024141230044

Name: 范奕



(1) For seven measured voltage-current data pairs of a Zener diode, Compare several fitting and interpolation methods and then judge which curve is more reasonable for the device characteristic.

(2) For Runge's function

$$f(x) = \frac{1}{1 + 16x^2},$$

Generate equally spaced nodes in $[-1, 1]$, construct polynomial interpolation, and compare the interpolant with the exact function for $n = 5, 7, 9, 17, 19, 21$ nodes.

(3) For

$$f(x) = x \sin(2\pi x + 1),$$

Repeat the same interpolation experiment on $[-1, 1]$ for $n = 7, 9, 17, 19, 21$ equally spaced nodes.

目录

1 Part 1. Source Code **3**

1.1 Problem 1: multi-method fitting for Zener-diode data 3

1.2 Problem 2: Runge function interpolation 4

1.3 Problem 3: interpolation of $f(x) = x\sin(2\pi x + 1)$ 6

2 Part 2. Theoretical Analysis **7**

2.1 Problem 1 7

2.2 Problem 2 8

2.3 Problem 3 8

3 Part 3. Numerical Results **9**

3.1 Problem 1: fitting comparison for the Zener diode 9

3.2 Problem 2: Runge function 10

3.3 Problem 3: interpolation of $x\sin(2\pi x + 1)$ 11

4 Part 4. Discussion **13**

4.1 Problem 1 13

4.2 Problem 2 13

4.3 Problem 3 14

1 Part 1. Source Code

This section keeps the structure of the template and records the core C programs I used in the three problems.

1.1 Problem 1: multi-method fitting for Zener-diode data

```
#include <stdio.h>
#include <math.h>
#define N 7

double X[N] = {-1.00, 0.00, 1.27, 2.55, 3.82, 4.92, 5.02};
double Y[N] = {-14.58, 0.00, 0.00, 0.00, 0.00, 0.88, 11.17};

/* 6th-order global polynomial interpolation */
double poly6_eval(double x) {
   int i, j;
   double sum = 0.0;
   for (i = 0; i < N; i++) {
      double term = Y[i];
      for (j = 0; j < N; j++) {
         if (j != i) term *= (x - X[j]) / (X[i] - X[j]);
      }
      sum += term;
   }
   return sum;
}

/* natural cubic spline coefficients */
void natural_spline_coeff(double a[], double b[], double c[], double
    d[]) {
   double h[N-1], alpha[N], l[N], mu[N], z[N];
   int i, j;
   for (i = 0; i < N - 1; i++) {
      h[i] = X[i+1] - X[i];
      a[i] = Y[i];
   }
   alpha[0] = alpha[N-1] = 0.0;
   for (i = 1; i < N - 1; i++) {
      alpha[i] = 3.0 / h[i] * (Y[i+1] - Y[i])
```

```
            - 3.0 / h[i-1] * (Y[i] - Y[i-1]);
   }
   l[0] = 1.0; mu[0] = z[0] = 0.0;
   for (i = 1; i < N - 1; i++) {
      l[i] = 2.0 * (X[i+1] - X[i-1]) - h[i-1] * mu[i-1];
      mu[i] = h[i] / l[i];
      z[i] = (alpha[i] - h[i-1] * z[i-1]) / l[i];
   }
   l[N-1] = 1.0; z[N-1] = c[N-1] = 0.0;
   for (j = N - 2; j >= 0; j--) {
      c[j] = z[j] - mu[j] * c[j+1];
      b[j] = (Y[j+1] - Y[j]) / h[j]
          - h[j] * (c[j+1] + 2.0 * c[j]) / 3.0;
      d[j] = (c[j+1] - c[j]) / (3.0 * h[j]);
   }
}

/* piecewise linear interpolation */
double linear_eval(double x) {
   int i;
   if (x <= X[0]) return Y[0];
   if (x >= X[N-1]) return Y[N-1];
   for (i = 0; i < N - 1; i++) {
      if (x >= X[i] && x <= X[i+1]) {
         double t = (x - X[i]) / (X[i+1] - X[i]);
         return Y[i] + t * (Y[i+1] - Y[i]);
      }
   }
   return 0.0;
}

int main(void) {
   printf("Problem 1 compares polynomial, spline, linear and PCHIP-
      type curves.\n");
   return 0;
}
```

1.2 Problem 2: Runge function interpolation

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <math.h>
#define SAMPLE_POINTS 2000

double runge_function(double x) {
   return 1.0 / (1.0 + 16.0 * x * x);
}

void generate_nodes(int n, double x[], double y[]) {
   int i;
   double h = 2.0 / (n - 1);
   for (i = 0; i < n; i++) {
      x[i] = -1.0 + i * h;
      y[i] = runge_function(x[i]);
   }
}

double lagrange_interpolation(double t, int n, double x[], double y
   []) {
   int i, j;
   double p = 0.0;
   for (i = 0; i < n; i++) {
      double L = 1.0;
      for (j = 0; j < n; j++) {
         if (j != i) L *= (t - x[j]) / (x[i] - x[j]);
      }
      p += y[i] * L;
   }
   return p;
}

int main(void) {
   int node_counts[] = {5, 7, 9, 17, 19, 21};
   int k;
   for (k = 0; k < 6; k++) {
      int n = node_counts[k];
      double *x = (double *)malloc(n * sizeof(double));
      double *y = (double *)malloc(n * sizeof(double));
      generate_nodes(n, x, y);
      printf("Runge case: n = %d, polynomial degree = %d\n", n, n -
         1);
```

```
      free(x);
      free(y);
   }
   return 0;
}
```

1.3 Problem 3: interpolation of $f(x) = x\sin(2\pi x + 1)$

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define PI 3.14159265358979323846

double target_function(double x) {
   return x * sin(2.0 * PI * x + 1.0);
}

void generate_nodes(int n, double x[], double y[]) {
   int i;
   double h = 2.0 / (n - 1);
   for (i = 0; i < n; i++) {
      x[i] = -1.0 + i * h;
      y[i] = target_function(x[i]);
   }
}

double lagrange_interpolation(double t, int n, double x[], double y
   []) {
   int i, j;
   double p = 0.0;
   for (i = 0; i < n; i++) {
      double L = 1.0;
      for (j = 0; j < n; j++) {
         if (j != i) L *= (t - x[j]) / (x[i] - x[j]);
      }
      p += y[i] * L;
   }
   return p;
}
```

```
int main(void) {
   int node_counts[] = {7, 9, 17, 19, 21};
   int k;
   for (k = 0; k < 5; k++) {
      printf("Problem 3 case: n = %d\n", node_counts[k]);
   }
   return 0;
}
```

2 Part 2. Theoretical Analysis

2.1 Problem 1

For the first problem, I do not treat it as a pure mathematical interpolation exercise only. It is also a simple modeling problem. From the seven measured pairs, I can already see the typical Zener-diode behavior: a reverse region with a large negative current, a broad blocking region where the current is close to zero, and then a sharp rise near forward conduction. Because of that, I think the most important question is not only whether a curve passes through all the given points, but also whether the curve still looks physically reasonable between the nodes.

The methods I compared are:

(1) global 6th-order polynomial interpolation;

(2) natural cubic spline interpolation;

(3) piecewise linear interpolation;

(4) shape-preserving cubic Hermite interpolation (PCHIP);

(5) a piecewise physics-inspired model.

Before plotting the results, my expectation was the following. A single high-order polynomial would probably oscillate too much. A natural spline should look smoother, but it might still overshoot when the local data change very fast. Piecewise linear interpolation should be stable, although it is not smooth. PCHIP should keep the local shape better than the natural spline. The physics-inspired piecewise model should match the operating regions of the diode best in a qualitative sense, but it still depends on how well the parameters are chosen.

2.2 Problem 2

For Runge's function

$$f(x) = \frac{1}{1 + 16x^2},$$

I use equally spaced nodes in $[-1, 1]$ and construct the unique interpolating polynomial through those nodes. If n nodes are used, then the interpolation polynomial has degree $n - 1$.

This problem is the classical example of the Runge phenomenon. Even though the interpolant agrees with the exact function at every node, increasing the degree with equally spaced nodes can produce very large oscillations near the two ends of the interval. So in this case, adding more equally spaced nodes does not necessarily improve the global approximation.

2.3 Problem 3

For

$$f(x) = x \sin(2\pi x + 1),$$

I again use equally spaced nodes and construct the Lagrange interpolating polynomial. This function is also oscillatory, but compared with the Runge function, its interpolation behavior should be much better on the same interval. So I expect the numerical results to improve steadily when the number of nodes becomes large enough.

The interpolation polynomial is written as

$$P_{n-1}(x) = \sum_{i=0}^{n-1} f(x_i) L_i(x),$$

where

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j}.$$

This formula guarantees that $P_{n-1}(x_i) = f(x_i)$ for all interpolation nodes.

3 Part 3. Numerical Results

3.1 Problem 1: fitting comparison for the Zener diode

Figure 1 shows all fitting and interpolation curves on the same axes. I changed the figure title to English so that all labels can be displayed correctly in the final report.

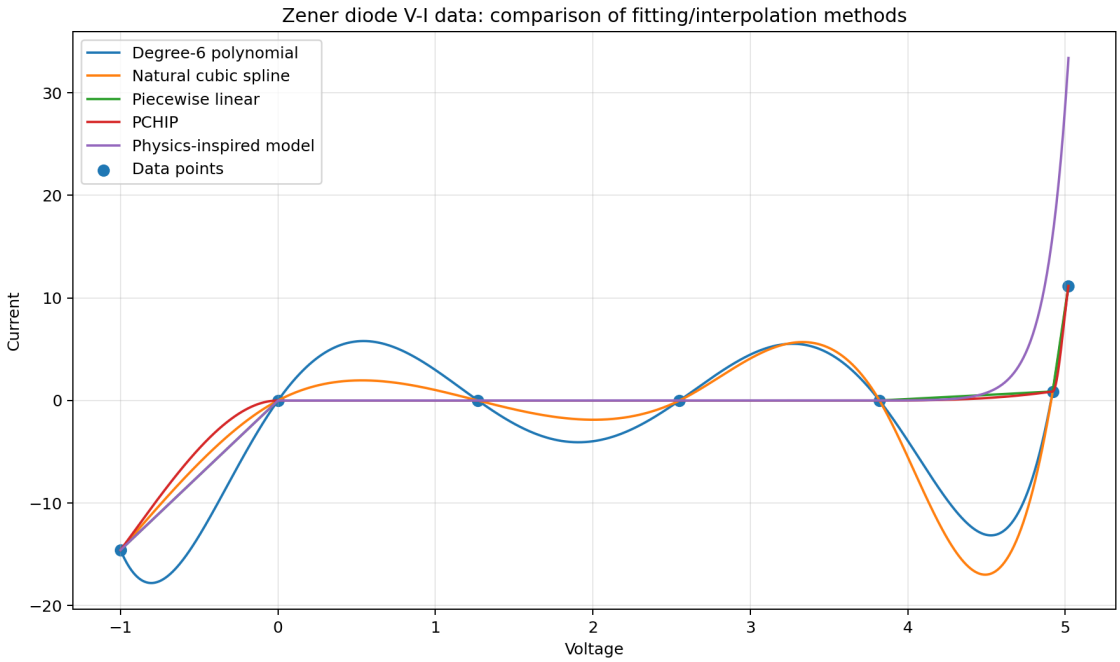


图 1: Comparison of several fitting and interpolation methods for the Zener-diode data.

I also list a few representative interpolated values in Table 1. These numbers are helpful because they show how differently the methods behave between the same measurement nodes.

表 1: Representative values from different methods in Problem 1.

x	global poly	spline	linear	PCHIP	piecewise model
-0.50	-13.1611	-6.0776	-7.2900	-4.6646	-7.2900
2.00	-3.9726	-1.8734	0.0000	0.0000	0.0000
4.50	-13.0993	-16.9724	0.5440	0.2423	0.7568
5.00	8.8125	9.0200	9.1120	8.6808	28.8720

3.2 Problem 2: Runge function

Figure 2 gives the exact function and the equally spaced polynomial interpolants for $n = 5, 7, 9, 17, 19, 21$ nodes.

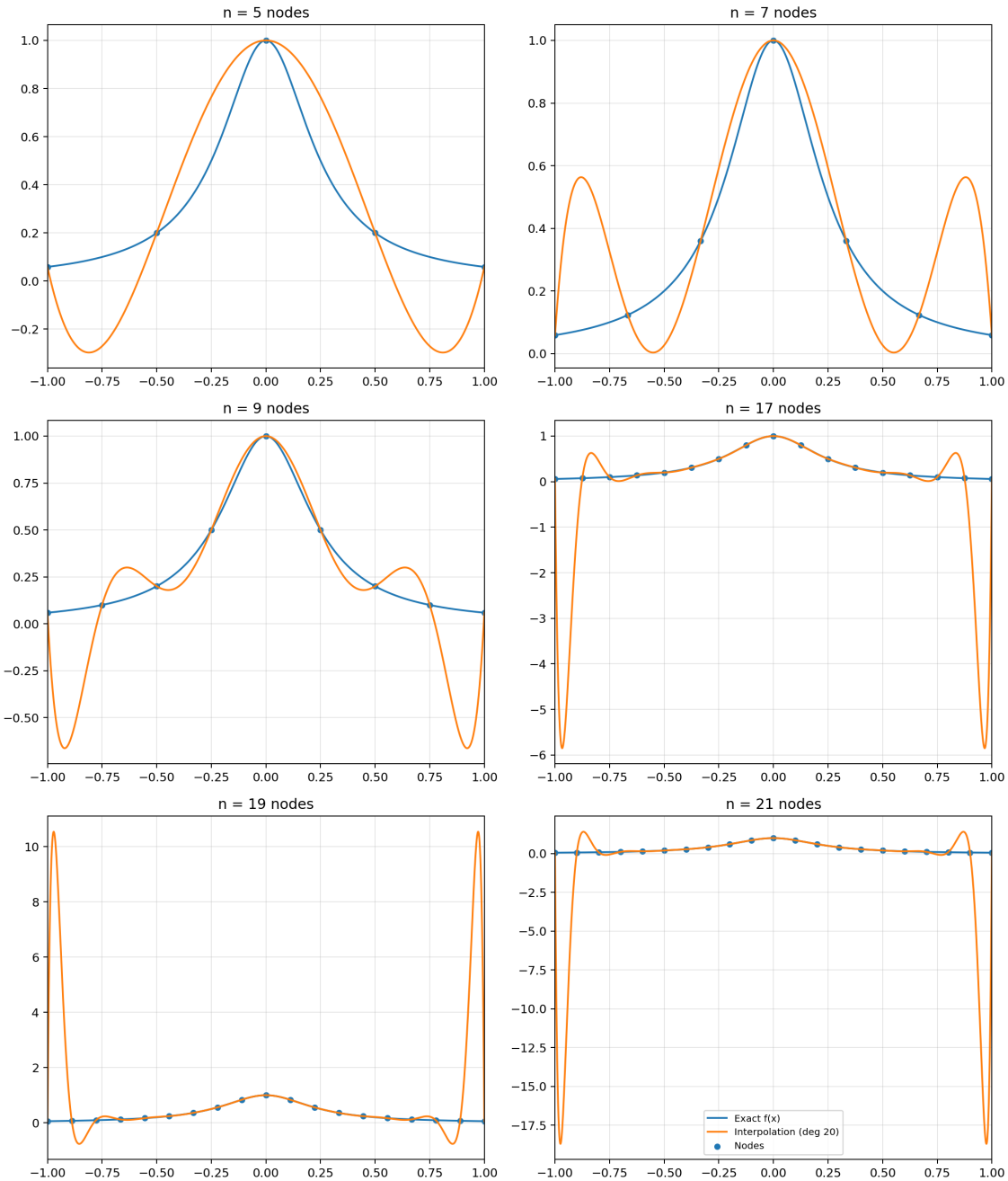


图 2: Runge function and polynomial interpolation with equally spaced nodes.

The maximum absolute errors on a dense grid are summarized in Table 2. From these values, I can directly see that the error grows again for larger n, which is exactly the Runge phenomenon.

表 2: Maximum absolute error for Problem 2.

Number of nodes n	max $\lvert P_{n-1}(x) - f(x)\rvert$
5	3.8530×10^{-1}
7	4.8888×10^{-1}
9	7.3189×10^{-1}
17	5.9132×10^{0}
19	1.0479×10^{1}
21	1.8768×10^{1}

3.3 Problem 3: interpolation of $x\sin(2\pi x + 1)$

Figure 3 shows the exact function and the interpolation curves for $n = 7, 9, 17, 19, 21$ nodes.

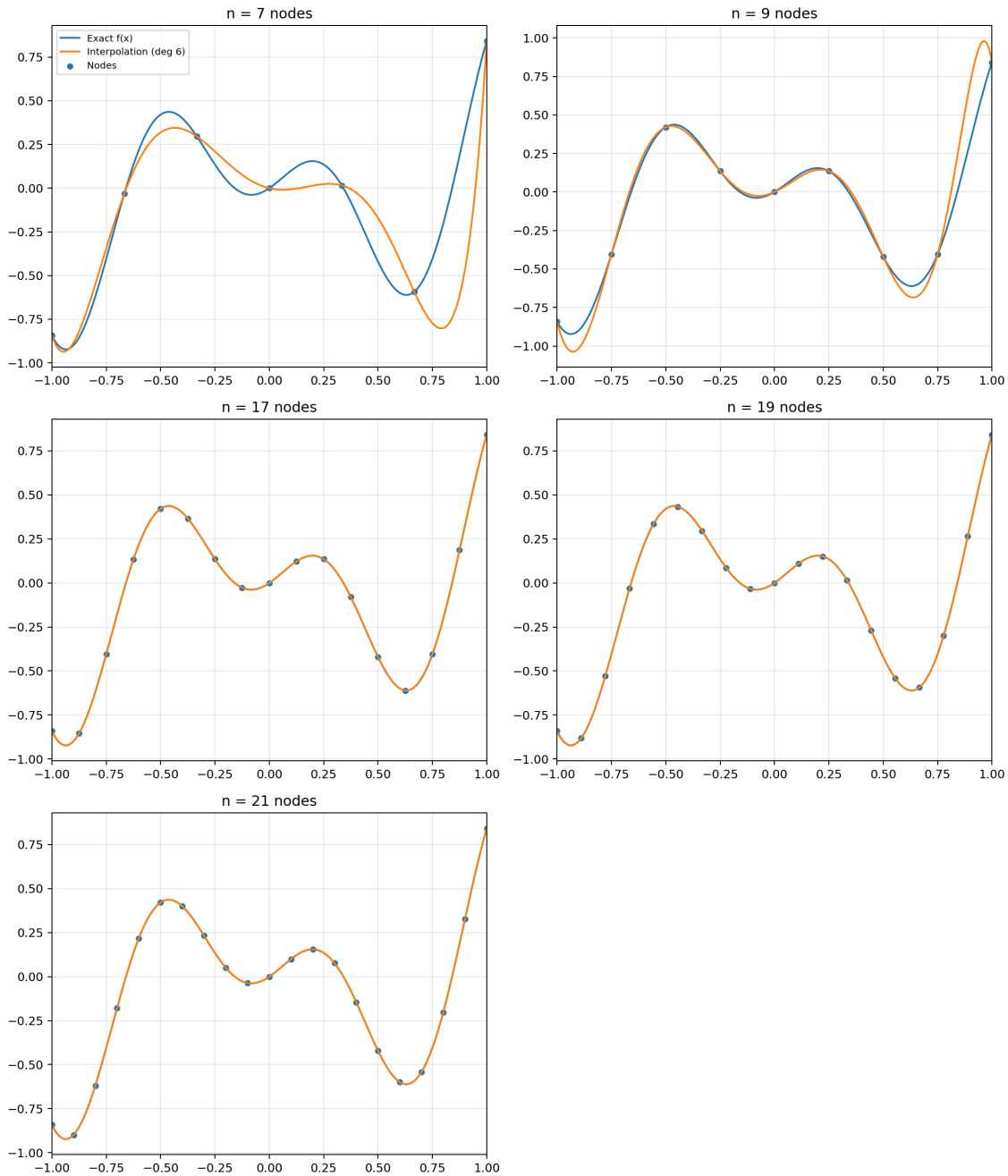


图 3: Interpolation of $f(x) = x\sin(2\pi x + 1)$ with equally spaced nodes.

The corresponding maximum absolute errors are listed in Table 3. In contrast to Problem 2, the error decreases rapidly when more nodes are used.

表 3: Maximum absolute error for Problem 3.

Number of nodes n	$\max \lvert P_{n-1}(x) - f(x) \rvert$
7	8.0490×10^{-1}
9	3.9831×10^{-1}
17	1.6216×10^{-4}
19	1.0882×10^{-5}
21	5.8700×10^{-7}

4 Part 4. Discussion

4.1 Problem 1

From the figure and the table, I think the global 6th-order polynomial and the natural cubic spline are both unsatisfactory as final representations of the Zener-diode data. They pass through the measured points, but they also create obvious nonphysical oscillations in regions where the current should stay near zero or should change monotonically. For example, near $x = 4.50$, the global polynomial predicts about -13.10 and the spline predicts about -16.97, which clearly do not match the local trend of the measurements.

The piecewise linear interpolant is numerically stable and does not introduce artificial oscillation, but it is not smooth. In my view, PCHIP gives the best compromise among the data-driven interpolation methods because it keeps the local shape and monotonicity while avoiding the severe overshoot of the natural spline. The piecewise physics-inspired model captures the qualitative operating regions of the diode, but without further parameter optimization it can still deviate a lot near the forward-conduction points. So for this dataset, I would choose PCHIP as the most reasonable final interpolation result.

4.2 Problem 2

Problem 2 reproduces the classical Runge phenomenon very clearly. When the number of equally spaced nodes increases from 5 to 21, the interpolation error near the interval endpoints becomes worse instead of better. Table 2 shows that the maximum error grows from about 3.85×10^{-1} to nearly 1.88×10^{1}. This means that a high-degree polynomial through equally spaced nodes is highly unstable for this function.

The most important lesson I learned from this problem is that interpolation quality depends not only on the number of nodes, but also on the node distribution and on the shape of the function itself. For Runge-type functions, equally spaced nodes are a poor choice for high-degree global interpolation. Better alternatives would be Chebyshev nodes or piecewise low-degree methods such as splines.

4.3 Problem 3

Problem 3 behaves very differently from Problem 2. Although the target function is oscillatory, it is still approximated increasingly well as the number of equally spaced nodes grows. The error drops from about 8.05×10^{-1} for $n = 7$ to about 5.87×10^{-7} for $n = 21$. In the figure, the interpolant with 17 or more nodes is already visually very close to the exact curve on almost the whole interval.

This comparison is important to me because it shows that equally spaced high-degree interpolation is not always disastrous. Its performance depends strongly on the specific function being approximated. Putting Problem 2 and Problem 3 together, I can see that the same interpolation algorithm may behave very badly for one function and very well for another.

End of Report